

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

2.2 Intents

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Índice

1. **Introducción**
2. Casos de uso principales
3. Resolución de un Intent
4. Estructura de un Intent
5. Tipos de Intents
6. Intents con resultados
7. Tasks and Back stack

Introducción

- Objeto de mensajería
- Comunicación entre componentes
    - Indica al componente que lo recibe qué hacer
- Puede encapsular datos para que los reciba el componente destino
- Viene a ser un “Android, haz esto, con estos datos”
- Se pueden reusar
- Lo utilizan Activities, Services, BroadcastReceivers, ContentProviders
- Siempre lo ejecuta el sistema: Android

Índice

1. Introducción
2. **Casos de uso principales**
3. Resolución de un Intent
4. Estructura de un Intent
5. Tipos de Intents
6. Intents con resultados
7. Tasks and Back stack

Casos de uso principales (I)

- Iniciar una actividad
  - *startActivity*(Intent intent, Bundle options)
    - El Intent indica la actividad que se debe iniciar
      - Contiene los datos necesarios para ello
    - El Bundle es un objeto que indica opciones para iniciar la Activity
      - Puede ser null
  - *startActivityForResult*(Intent intent, int requestCode)
    - La Activity origen espera un resultado

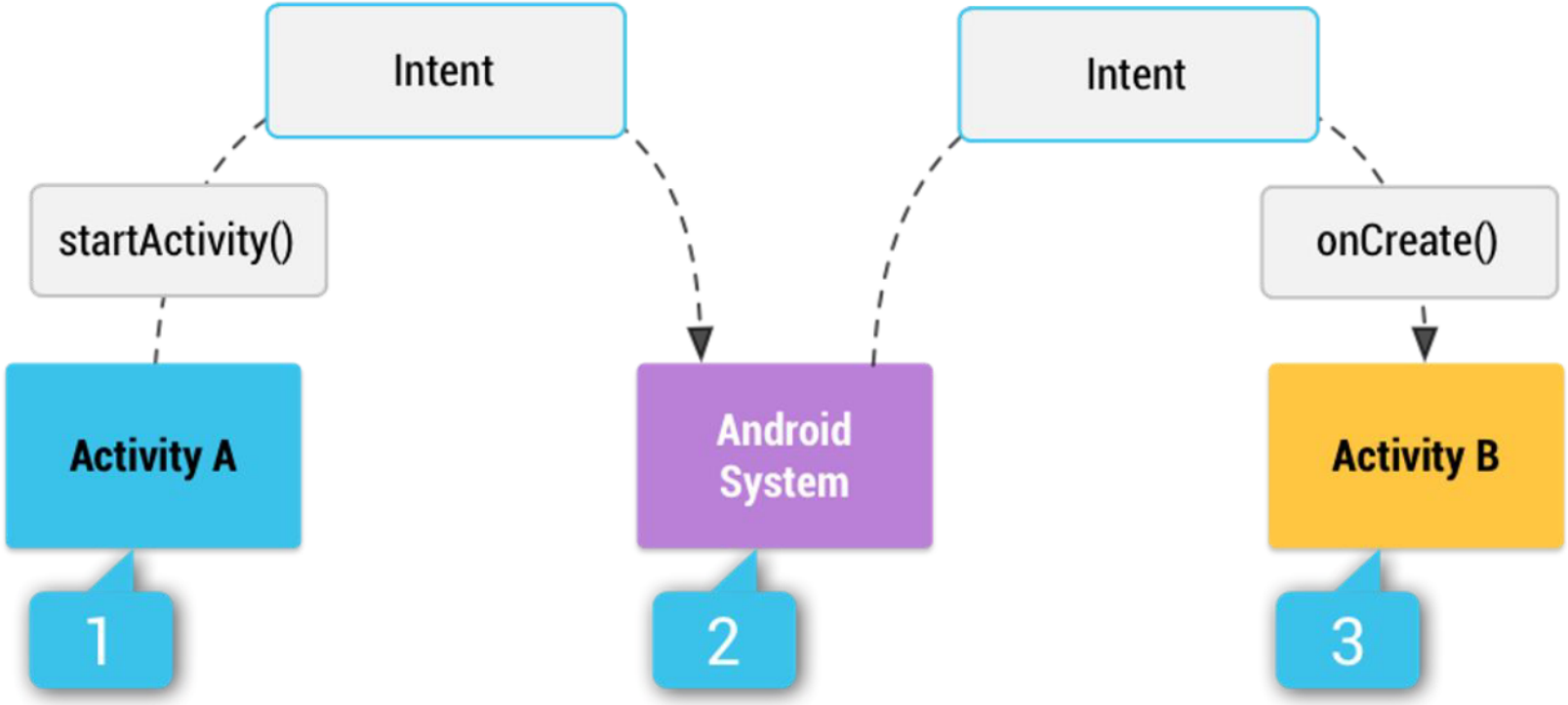
Casos de uso principales (y II)

- Iniciar un Servicio (p.e. *startService* o *bindService*)
  - Un servicio es un componente que realiza operaciones en segundo plano
  - Desde Android 5.0 (API 21), también se utiliza el *JobScheduler*
- Iniciar una emisión (Broadcast) (p.e. *sendBroadcast*)
  - Un broadcast es un aviso que cualquier aplicación puede enviar y recibir
  - Hay broadcasts del sistema
    - Cuando el dispositivo se conecta a una wifi
    - Batería baja
    - ...

Índice

1. Introducción
2. Casos de uso principales
3. **Resolución de un Intent**
4. Estructura de un Intent
5. Tipos de Intents
6. Intents con resultados
7. Tasks and Back stack

Resolución de un Intent (I)



Resolución de un Intent (y II)

- La actividad A crea un Intent con una descripción de acción y la pasa a *startActivity*()
- El sistema Android busca en todas las apps un filtro de Intents que coincida con la acción indicada
- Cuando se encuentra una coincidencia, el sistema inicia la actividad encontrada (actividad B) llamando a su método *onCreate*() y pasándole el Intent
  - Puede haber varias coincidencias
  - El sistema te da a elegir qué aplicación quieres que se abra

Índice

1. Introducción
2. Casos de uso principales
3. Resolución de un Intent
4. **Estructura de un Intent**
5. Tipos de Intents
6. Intents con resultados
7. Tasks and Back stack

Estructura de un Intent (I)

- El componente destino
  - Es opcional, pero si necesitas iniciar un componente específico en tu aplicación debes especificarlo (explícito)
  - Normalmente va a ser el nombre de la clase que implementa la Activity
- La acción
  - String que especifica la acción genérica que se debe realizar
  - Puedes usar acciones ya creadas del sistema o acciones propias

```
static final String ACTION_CHECK_INTERNET = "com.example.action.CHECK_INTERNET";
```

Estructura de un Intent (II)

- Ejemplo de acciones más comunes
  - ACTION_VIEW
    - Se usa para pasar información que la actividad pueda mostrar al usuario
    - Fotos, dirección de maps, URLs, etc ...
  - ACTION_SEND
    - Se usa para que el usuario pueda compartir información mediante otra app
    - Mandar correo, publicar en redes sociales, etc ...

Estructura de un Intent (III)

- Datos
  - Una URI es un objeto (Uri) que hace referencia
    - a los datos en los que se debe realizar la acción o
    - el tipo de MIME de esos datos
      - image/gif, image/jpeg, video/mp4, text/html, text/plain, application/vnd.google-earth.kml+xml
      - Normalmente el MIME va acorde con la acción
    - Especificar el tipo de MIME ayuda a Android a encontrar el mejor componente para recibir el Intent

Estructura de un Intent (IV)

- Ejemplos de URIs
  - Tel:034981167000
  - geo:43.332690, -8.410795?z=16
  - http://www.fic.udc.es
  - content://contacts/people
  - market://details?id=package
- Ejemplos de MIME types
  - image/gif; image/jpeg; video/mp4; text/html; text/plain; application/vnd.google-earth.kml+xml

Estructura de un Intent (V)

- Categoría
  - String que contiene información adicional sobre el tipo de componente
    - Se pueden incluir las que se quieran
    - La mayoría no requieren una categoría
  - Categorías comunes
    - CATEGORY_BROWSABLE
      - Permite que se inicie un navegador web para mostrar los datos
    - CATEGORY_LAUNCHER
      - Es la Activity con la que se va a iniciar la app
      - Sale en el selector de aplicación

Estructura de un Intent (VI)

- Extras
  - Pares clave-valor (Bundle) con información adicional necesaria para completar la acción solicitada
  - Algunas acciones usan extras específicos
    - A la acción ACTION_SEND se le puede especificar el destinatario con la clave EXTRA_EMAIL y el asunto con EXTRA_SUBJECT
  - También se permite definir Extras propios

```
static final String EXTRA_NUMBER_OF_ITEMS = "com.example.action.NUMBER_OF_ITEMS";
```

Estructura de un Intent (y VII)

- Flags
  - Los indicadores (o *flags*) son metadatos del Intent
  - Los flags pueden indicar al sistema Android cómo iniciar una actividad:
    - a qué tarea debe pertenecer
    - cómo tratarla después de que se inicia
  - Ejemplos
    - FLAG_ACTIVITY_NEW_TASK
    - FLAG_ACTIVITY_CLEAR_TOP
    - FLAG_ACTIVITY_SINGLE_TOP

Índice

1. Introducción
2. Casos de uso principales
3. Resolución de un Intent
4. Estructura de un Intent
5. **Tipos de Intents**
6. Intents con resultados
7. Tasks and Back stack

Tipos de Intents (I)

- Explícitos
  - Intent que indica un componente específico de la aplicación

```
Intent userProfileIntent = new Intent(this, UserProfileActivity.class);
startActivity(userProfileIntent);
```

- Implícitos
  - Intent que especifica una acción que puede invocar cualquier aplicación en el dispositivo que puede realizar la acción
    - Útil cuando la aplicación no puede realizar la acción, pero otras aplicaciones probablemente sí

Tipos de Intents (y II)

- Cómo recibir un Intent implícito
  - Si tu app puede abrir ciertas acciones, tienes que “avisar al sistema”
    - Esto es, explicitarlo en el AndroidManifest.xml

```
<activity android:name="ShareActivity">
   <intent-filter>
       <action android:name="android.intent.action.SEND"/>
       <category android:name="android.intent.category.DEFAULT"/>
       <data android:mimeType="text/plain"/>
   </intent-filter>
</activity>
```

¿Qué indica esta parte de un AndroidManifest?

```
<activity android:name="MainActivity">
   <intent-filter>
       <action android:name="android.intent.action.MAIN" />
       <category android:name="android.intent.category.LAUNCHER" />
   </intent-filter>
</activity>
<activity android:name="ShareActivity">
   <intent-filter>
       <action android:name="android.intent.action.SEND"/>
       <category android:name="android.intent.category.DEFAULT"/>
       <data android:mimeType="text/plain"/>
   </intent-filter>
   <intent-filter>
       <action android:name="android.intent.action.SEND"/>
       <action android:name="android.intent.action.SEND MULTIPLE"/>
       <category android:name="android.intent.category.DEFAULT"/>
       <data android:mimeType="application/vnd.google.panorama360+jpg"/>
       <data android:mimeType="image/*"/>
       <data android:mimeType="video/*"/>
   </intent-filter>
</activity>
```



Intent implícitos

- Al crear un intent implícito se pueden incluir ciertas restricciones y/o datos

```
Intent intentSend = new Intent(ACTION_SEND);
intentSend.setType("text/plain");
intentSend.putExtra(Intent.EXTRA_TEXT, " my text to send");
startActivity(intentSend);
```

- También se puede mostrar una lista de las apps instaladas que pueden ejecutar un determinado Intent implícito:

```
Intent intentShareApps = Intent.createChooser(intentSend, null);
startActivity(intentShareApps);
```

Índice

1. Introducción
2. Casos de uso principales
3. Resolución de un Intent
4. Estructura de un Intent
5. Tipos de Intents
6. **Intents con resultados**
7. Tasks and Back stack

Intent con resultado (I)

- Intent que abre un componente pero que espera que este les dé un resultado
- El intent se lanza con
  - *startActivityForResult(Intent intent, int requestCode)*
  - Donde *requestCode* es un entero que nos servirá para distinguir los resultados que nos mandarán los Intents destino

```
private static final int ACTION_RECOGNIZE_SPEECH = 83838;
[...]
Intent intent = new Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH);
intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
RecognizerIntent.LANGUAGE_MODEL_FREE_FORM);
startActivityForResult(intent, requestCode);
```

Intent con resultado (II)

- Y los datos se reciben en el método
  - *protected void onActivityResult(int requestCode, int resultCode, Intent data)*
  - Donde *resultCode* es el mismo que hemos puesto en *startActivityWithResult*
  - Y *resultCode* es un código que nos dice cómo fue la operación

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
   if (requestCode == ACTION_RECOGNIZE_SPEECH && resultCode == RESULT_OK) {
       // Recoger datos de "data" y utilizarlos
   }
}
```

Intent con resultado (III)

- El componente destino (Activity B) escribe el resultado con los métodos
  - public final void *setResult*(int resultCode)
  - public final void *setResult*(int resultCode, Intent data)
  - Donde resultCode es un entero que indica cómo fue la operación
    - Se suelen utilizar los predefinidos
      - Activity.RESULT_OK - La operación salió bien (-1)
      - Activity.RESULT_CANCELED - La operación salió mal (0)
      - Activity.RESULT_FIRST_USER - Devolver otro valor (+1)

Intent con resultado (y IV)

- El componente llamado (p.e. actividad) puede obtener los datos del Intent

```
Bundle bundle = getIntent().getExtras();
if (bundle != null) {
   String text = bundle.getString(KEY_TEXT, "mi dato de inicio"));
   textView.setText(text);
}
```

- También debe confeccionar el Intent del resultado

```
Intent intent = new Intent();
intent.putExtra(KEY_TEXT, "mi dato resultado");
setResult(RESULT_OK, intent);
finish();
```

Intent con resultado (New way)

- Desde API 30 ha cambiado el intent con resultado (evita los códigos numéricos):
  - `my_startActivityForResult.launch(intentContinuar);`

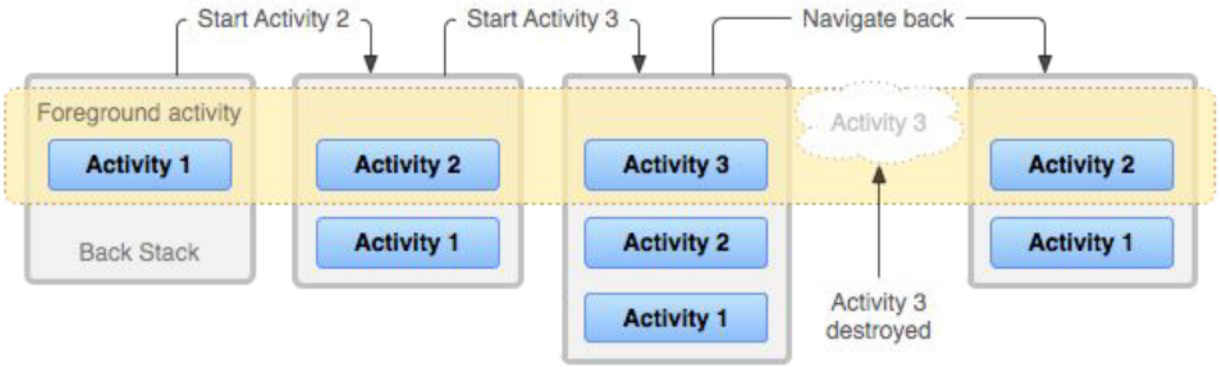
```
ActivityResultLauncher<Intent> my startActivityForResult = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == AppCompatActivity.RESULT_OK) {
            Intent data = result.getData();
            Bundle bundle = data.getExtras();
            if (bundle!= null) {
                String text = bundle.getString(KEY TEXT, getString(R.string.emptyField));
                Toast.makeText(getApplicationContext(), text, Toast.LENGTH_SHORT).show();
            }
        }
    }
);
```

Índice

1. Introducción
2. Casos de uso principales
3. Resolución de un Intent
4. Estructura de un Intent
5. Tipos de Intents
6. Intents con resultados
7. **Tasks and Back stack**

Tasks and Back stack (I)

- Tarea (o *task*) es una agrupación de actividades con la que interactúan los usuarios cuando realizan una acción determinada
- Pila (o *stack*) es la estructura de datos donde se organizan las actividades
  - Pila -> LIFO

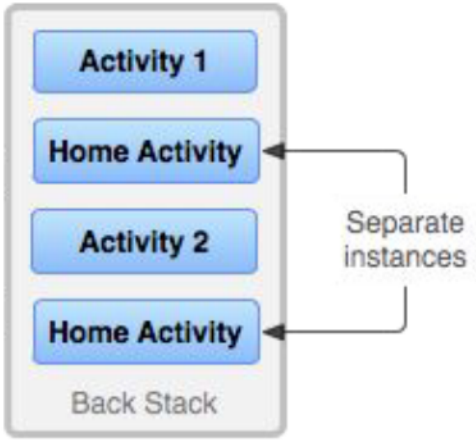


Tasks and Back stack (II)

- Una tarea puede pasar a segundo plano (*background*)
  - Cuando se inicia una nueva tarea
  - Cuando se presiona el botón Inicio para mostrar la pantalla principal
- En segundo plano se detienen todas las actividades
  - La pila de actividades de la tarea permanece intacta
  - Cada tarea tiene su pila de actividades
- Las actividades de la pila de actividades nunca se reordenan

Tasks and Back stack (III)

- Si tu app puede iniciar actividades desde más de una actividad
  - Se crea una nueva instancia de esa actividad
  - Se inserta en la pila
    - En lugar de colocar una instancia previa de la actividad en la parte superior
  - Esto permite crear instancias de una actividad en tu app varias veces
    - Incluso desde diferentes tareas



Tasks and Back stack (y IV)

- Cuando una actividad inicia otra, el sistema retiene su estado para posteriormente poder continuar su ejecución
- Cuando un usuario sale de una tarea con el botón de Home, la actividad se para y se va a segundo plano (*background*)
- Si el usuario pulsa el botón de atrás, la actividad se destruye y el sistema no guarda el estado
    - La actividad previa en el *back stack* (de haberla), continúa
- Esto está fuertemente relacionado con el diseño de navegación de una app

Referencias

- https://developer.android.com/guide/components/intents-filters
- https://developer.android.com/reference/android/content/Intent.html
- https://developer.android.com/guide/components/activities/tasks-and-back-stack
- https://developer.android.com/guide/navigation/navigation-design-graph